Table of Contents

..... 1
Handle optional inputs 1
Initialize path and files names 2
Load data from FAST run 2
Run Simulink simulations or load data 3
Create output plots (for use in power point) 3
Figure for paper 4


```
function runCompareModels(strWindType,loadData,figNo1,yAxCell)

% runCompareModels compares two Simulink models with FASTTool
 simulation
% data generated with the baseline controller.
%
% All inputs are optional:
% - strWindType: Two testcases: step sweep 4 to 25 ms and normal dist.
 with
%   18 m/s mean (Default: Sweep)
% - loadData: load simulation output data if available instead of
 running
%   simulation (Default: 1)
% - figNo1: Number of figure (Default: 1)
% - yAxCell: Axes labels
```

Handle optional inputs

The default inputs are provided here.

```
% Two testcases: step sweep 4 to 25 ms and normal dist. 18 m/s mean
if ~nargin || isempty(strWindType)
   strWindType = 'Sweep';
end

if nargin < 2 || isempty(loadData)
    loadData = 0; %load simulation output data if available;
end

if nargin < 3 || isempty(figNo1)
    figNo1 = 1;
end
figNo2 = figNo1 + 1;

if nargin < 4 || isempty(yAxCell) % Axes labels for figure
    yAxCell = {'wind V [m/s]', 'GenTq T_g [kNm]', 'Pitch \beta
 [°]', 'RotSpd \omega_r [rpm]',...
    'GenPwr P_g [MW]','Twr_{FA} y_t [m/s^2]'};
end
```

Initialize path and files names

The path to figure and input directory is set. The name of the Simulink models for Mdl1 and Mdl2 are provided. The name of the mat file with the FAST reference data is given.

```
% Set path to inputdata directory
workDir = fileparts(mfilename('fullpath'));
mainDir = fileparts(workDir);
dataInDir = fullfile( mainDir,'dataIn');
addpath(dataInDir);

% Set path to figure directory
figDir = fullfile(mainDir,'figDir');
if ~isfolder(figDir)
    mkdir(figDir)
end

% Set path to data output directory
dataDirOut = fullfile(workDir, 'dataOut');
if ~isfolder(dataDirOut)
    mkdir(dataDirOut)
end

% Provide names of Simulink models to be run
simMdlname1 = 'test_SimulinkMdl1_Baseline';
simMdlname2 = 'test_SimulinkMdl2_Baseline';

% Provide names of FAST simulation data to be loaded
if strcmp(strWindType,'Sweep') == 1 % sweep from 4 to 25 in steps
    outDataSimulationMat = 'OutDataSweep.mat';
    strFig = '';
    testCaseStr = 'Wind Sweep';
else  % wind with average 18 m/s
    outDataSimulationMat = 'OutDataWind18NTW.mat';
    strFig = 'NTW18';
    testCaseStr = 'Wind, mean 18 m/s';
end
```

Load data from FAST run

Load FASTtool simulation data from dataIn folder.

```
load(outDataSimulationMat ,'OutTable');%

% Assign data from FAST to variables
idxTime = 1: height(OutTable);
OutTable = OutTable(idxTime,:);
OutTable.BldPitch1 = OutTable.BlPitch1;
OutTable.Torque = OutTable.GenTq;

% Calculate and assign wind amplitude from comonents
idxWind = contains(OutTable.Properties.VariableNames, 'Wind');
vectWind = OutTable{:,idxWind};
```

```
vectAmpWind = sqrt(sum((vectWind.^2),2));
```

Run Simulink simulations or load data

```
% Names of simulation output in order of Simulink bus
varnames =
 {'Wind', 'RotSpeed', 'GenPwr', 'GenTq', 'BlPitch1', 'NcIMUTAxs', 'NcIMUTAys'};

% Run simulation for two models or load mat files if available
matFileOutTableTest1 = fullfile(dataDirOut,
['OutTableTest1',strFig,'.mat']);
OutTableTest1 =
 getSimulationOutputTable(matFileOutTableTest1,loadData,OutTable,simMdlname1,varna

matFileOutTableTest2 = fullfile(dataDirOut,
['OutTableTest2',strFig,'.mat']);
OutTableTest2 =
 getSimulationOutputTable(matFileOutTableTest2,loadData,OutTable,simMdlname2,varna

Undefined function 'getSimulationOutputTable' for input arguments of
 type 'table'.

Error in runCompareModels (line 96)
OutTableTest1 =
 getSimulationOutputTable(matFileOutTableTest1,loadData,OutTable,simMdlname1,varna
```

Create output plots (for use in power point)

Two plots are created: 1st plot shows wind, rotor speed,

```
% Get color map for 'title legends'
cl = colormap('lines');
titleStr = [testCaseStr,': Mdl1: Rot+Twr {\color[rgb]
{',num2str(cl(1,:)),'}Mdl2: Rot,Twr,Bld+Act ',...
    '\color[rgb]{',num2str(cl(2,:)),'}FAST} '];

% Create output plot time reference
idxTime = 1: min([length(idxTime),
 height(OutTableTest1),height(OutTableTest2)]);
time = OutTable.Time(idxTime);
idxTime = time > 15;
time = time(idxTime);

% Change line width
defaultLineWidth = get(groot,'defaultLineLineWidth');
set(groot,'defaultLineLineWidth',0.5);

% 1st figure
figure(figNo1);
axPlot(1) = subplot(3,1,1);
plot(time,vectAmpWind(idxTime),time,OutTableTest1.Wind(idxTime),'--');
axis tight; grid on;
ylabel('wind [m/s]')
```

```
title(titleStr);

axPlot(2) = subplot(3,1,2);
plot(time,OutTableTest2.RotSpeed(idxTime),
 time,OutTable.RotSpeed(idxTime),time,OutTableTest1.RotSpeed(idxTime),'k--');
axis tight; grid on;
ylabel('RotSpd \omega_r [rpm]')

axPlot(3) = subplot(3,1,3);
plot(time,OutTableTest2.GenPwr(idxTime)/1000,time,OutTable.GenPwr(idxTime)/1000,ti
axis tight; grid on;
ylabel('GenPwr P_g [MW]')
xlabel('time [s]')
linkaxes(axPlot,'x');
set(findall(gcf,'-property','FontSize'),'FontSize',11)
set(gcf,'Name',['cmpTimeDomain_Wind',strFig])

print(fullfile(figDir,['cmpTimeDomain_Wind',strFig]), '-dpng');

% 2nd figure
figure(figNo2)
axPlot2(1) = subplot(3,1,1);
plot(time,OutTableTest2.BlPitch1(idxTime),time,OutTable.BlPitch1(idxTime),time,Out
axis tight; grid on;
ylabel('BldPitch1 \beta [rad]')
title(titleStr);

axPlot2(2) = subplot(3,1,2);
plot(time,OutTableTest2.GenTq(idxTime)/10^3,time,OutTable.GenTq(idxTime),time,OutT
axis tight; grid on;
ylabel('GenTq T_g [kNm]')

axPlot2(3) = subplot(3,1,3);
plot(time,OutTableTest2.NcIMUTAxs(idxTime),time,OutTable.NcIMUTAxs(idxTime),time,O
axis tight; grid on;
ylabel('Twr_{FA} y_t[m/s^2]')
xlabel('time [s]')
set(findall(gcf,'-property','FontSize'),'FontSize',11)
linkaxes(axPlot2,'x');
set(gcf,'Name',['cmpTimeDomain_BlPitchTqTwr',strFig])

print(fullfile(figDir,['cmpTimeDomain_BlPitchTqTwr',strFig]), '-
dpng');
```

Figure for paper

```
figure(figNo2+1)
axPlotAll(1) = subplot(6,1,1);
plot(time,vectAmpWind(idxTime),time,OutTableTest1.Wind(idxTime),'--');
axis tight; grid on;
ylabel(yAxCell{1}); %'wind [m/s]')
title(titleStr);
```

```
axPlotAll(4) = subplot(6,1,4);
plot(time,OutTableTest2.RotSpeed(idxTime),
 time,OutTable.RotSpeed(idxTime),time,OutTableTest1.RotSpeed(idxTime),'k--');
axis tight; grid on;
ylabel(yAxCell{4}); %'RotSpd \omega_r [rpm]')

axPlotAll(2) = subplot(6,1,2);
plot(time,OutTableTest2.GenTq(idxTime)/10^3,time,OutTable.GenTq(idxTime),time,OutT
axis tight;
posAxis = axis;
axis([posAxis(1:2), min(43,posAxis(3)), 44]);
ylabel(yAxCell{3}); %'T_g [kNm]')
grid on; % axis tight;
%grid on;
ylabel(yAxCell{2}); %'GenTq T_g [Nm]')

axPlotAll(3) = subplot(6,1,3);
plot(time,OutTableTest2.BlPitch1(idxTime),time,OutTable.BlPitch1(idxTime),time,Out
axis tight; grid on;
ylabel(yAxCell{3}); %'Pitch \beta [°]')

axPlotAll(5) = subplot(6,1,5);
plot(time,OutTableTest2.NcIMUTAxs(idxTime),time,OutTable.NcIMUTAxs(idxTime),time,O
axis tight; grid on;
ylabel(yAxCell{6}); %'Twr_{FA} y_t[m/s^2]')

axPlotAll(6) = subplot(6,1,6);
plot(time,OutTableTest2.GenPwr(idxTime)/1000,time,OutTable.GenPwr(idxTime)/1000,ti
axis tight; grid on;
ylabel(yAxCell{5}); %'GenPwr P_g [MW]')
xlabel('time [s]')
linkaxes(axPlotAll,'x');

set(gcf,'Name',['cmpTimeDomain_All',strFig])
posDefault = get(gcf, 'position');
set(gcf, 'position', [posDefault(1:3),posDefault(4)*1.5]);
set(groot,'defaultLineLineWidth',defaultLineWidth);

print(fullfile(figDir,['cmpTimeDomain_All',strFig]), '-dpng');
print(fullfile(figDir,['cmpTimeDomain_All',strFig]), '-depsc');
```

Published with MATLAB® R2019b